Simulazione di una rete SDN e mitigazione di attacchi DoS tramite controller Ryu.

Negli ultimi anni l'espansione di internet e delle reti ha creato sfide cruciali in termini di sicurezza. Ad oggi sulla rete è possibile fare di tutto: se da un lato questo permette di velocizzare i processi e superare la burocrazia, dall'altro l'iper-digitalizzazione espone a nuovi rischi che, se non affrontati a dovere, possono causare danni a utenti, aziende o addirittura nazioni intere. Le reti sono di vitale importanza per i cittadini, per le grandi organizzazioni e per gli apparati governativi, con volumi di traffico globale in ininterrotto aumento. Le sfide tecnologiche conseguenti sono molteplici e spaziano dalla velocità di trasmissione all'affidabilità, includendo obiettivi chiave come la Quality of Service (QoS), la persistenza dei dati e, non ultima, la loro sicurezza.

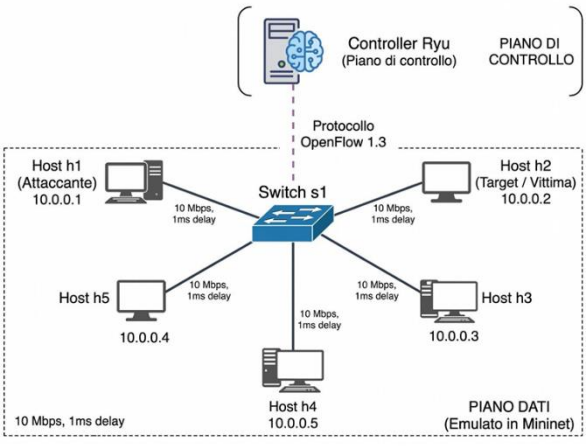
Il presente progetto si pone l'obiettivo di esplorare le potenzialità delle reti SDN applicate alla sicurezza informatica, concentrandosi sul rilevamento e sulla mitigazione degli attacchi Denial of Service (DoS). Attraverso l'utilizzo dell'emulatore Mininet e lo sviluppo di un'applicazione dedicata sul controller Ryu, verrà illustrato come il piano di controllo possa monitorare attivamente il traffico di rete e rilevare anomalie volumetriche. In caso di attacco, il sistema interviene autonomamente sfruttando il protocollo OpenFlow per istruire il piano dati, isolando istantaneamente l'indirizzo IP della fonte malevola e garantendo la totale continuità del servizio per gli utenti legittimi.

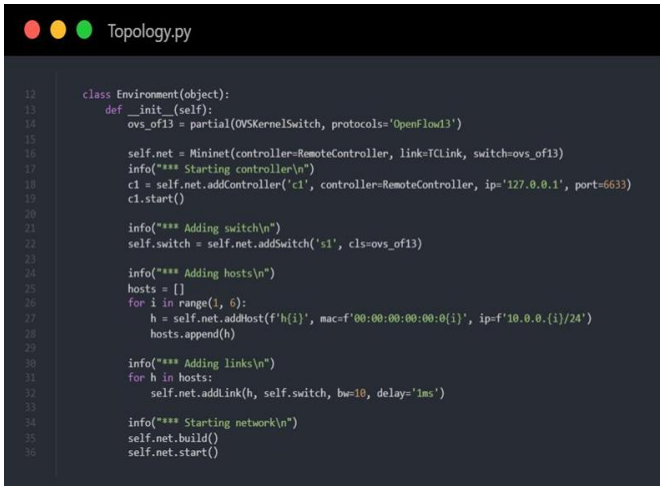
Si definisce attacco DoS una tipologia di attacco che punta a saturare le risorse di un server tramite l'utilizzo di semplici pacchetti di dati. L'attaccante genera un gran numero di richieste di connessione o di pacchetti malevoli ad altissima velocità, se l'infrastruttura non è in grado di filtrare tempestivamente questo traffico anomalo, le risorse si esauriscono, rendendo la macchina attaccata inutilizzabile e isolandola dagli altri nodi. Ciò si può tradurre in una mancanza di servizi per i client legittimi della rete.

OpenFlow è lo standard di comunicazione fondamentale nelle architetture SDN, progettato per permettere al piano di controllo centralizzato di dialogare direttamente con il piano dati dei dispositivi di rete. Funziona come un'interfaccia che consente al controller di programmare dinamicamente le tabelle di flusso (flow table) degli switch fisici o virtuali, dettando regole hardware su come i pacchetti debbano essere instradati o, in caso di attacco, scartati

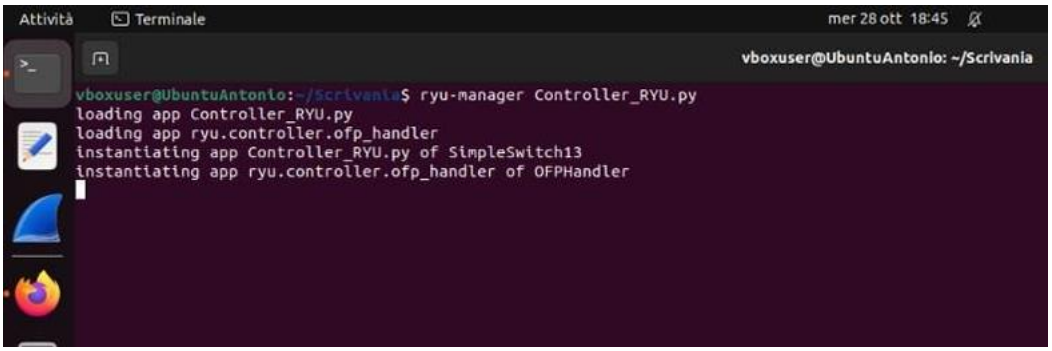
Ryu è un framework open source basato su Python che agisce come sistema operativo centrale (il "cervello") della rete SDN. Fornisce librerie e API native per gestire i protocolli di rete e ascoltare eventi asincroni, permettendo agli sviluppatori di creare applicazioni logiche personalizzate che analizzano il traffico 5 e inviano istruzioni decisionali agli switch sottostanti sfruttando il protocollo OpenFlow.

Posso eseguire lo script topology.py con il comando `sudo python3 topology.py`

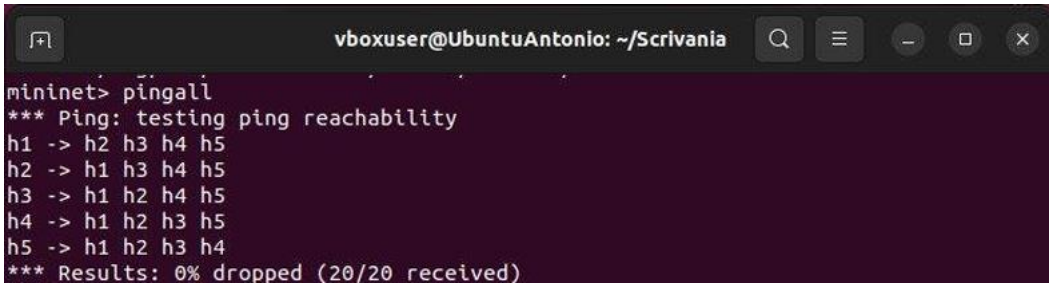




Il secondo passo è l'installazione del controller RYU che avviene tramite il seguente comando ryu-manager Controller_RYU.py



Esempio di test `pingall`



Lo script del Controller Ryu è di fondamentale importanza per gli obiettivi progettuali, in quanto realizza una logica software-defined per il controllo centralizzato della rete a 360°. La grande forza delle reti SDN moderne è proprio quella di poter definire le policy di instradamento e sicurezza attraverso righe di codice, permettendo di validare architetture complesse in simulazione prima della loro messa in opera. Mentre lo script della topologia si limitava a creare l'infrastruttura fisica (host, link e switch), questo modulo implementa l'intelligenza necessaria affinché i nodi possano effettivamente comunicare. Nello specifico, lo script si occupa di apprendere dinamicamente i percorsi (MAC learning), gestire l'inoltro dei pacchetti e, come obiettivo cruciale, rilevare e mitigare proattivamente eventuali attacchi DoS. Per l'implementazione del piano di controllo, il progetto prende come base di partenza l'applicazione ufficiale simple_switch_13.py, fornita nativamente dal framework Ryu. Questo script di base implementa le funzionalità standard di un Learning Switch di livello 2 compatibile con OpenFlow 1.3, occupandosi esclusivamente dell'apprendimento degli indirizzi MAC (MAC learning) e dell'inoltro basilare dei pacchetti. Tuttavia, al fine di raggiungere gli obiettivi di sicurezza prefissati, il codice originale è stato esteso introducendo un modulo di difesa personalizzato. Nello specifico, è stata aggiunta una logica di monitoraggio del traffico IPv4 entrante per tracciare il numero di pacchetti per singolo IP sorgente, rilevare saturazioni anomale tipiche degli attacchi DoS e intervenire dinamicamente installando regole di blocco ad alta priorità sullo switch.

Controller_RYU.py — python

```python
class SimpleSwitch13(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SimpleSwitch13, self).__init__(*args, **kwargs)
        self.mac_to_port = {}
        self.packet_count = {}
        self.DOS_THRESHOLD = 250  # Soglia di pacchetti per rilevare l'attacco DoS

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        # Installazione della regola table-miss di default
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)
        self.logger.info("Switch connesso e regola table-miss installata: DPID = %s", datapath.id)

    def add_flow(self, datapath, priority, match, actions, buffer_id=None):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        if buffer_id:
            mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                    priority=priority, match=match,
                                    instructions=inst)
        else:
            mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                    match=match, instructions=inst)
        datapath.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
        if ev.msg.msg_len < ev.msg.total_len:
            self.logger.debug("packet truncated: only %s of %s bytes",
                              ev.msg.msg_len, ev.msg.total_len)
        msg = ev.msg
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port']

        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocols(ethernet.ethernet)[0]

        if eth.ethertype == ether_types.ETH_TYPE_LLDP:
            return

        # Controllo e mitigazione DoS basato su indirizzo IP sorgente
        if eth.ethertype == ether_types.ETH_TYPE_IP:
            ip_pkt = pkt.get_protocol(ipv4.ipv4)
            src_ip = ip_pkt.src

            self.packet_count[src_ip] = self.packet_count.get(src_ip, 0) + 1

            if self.packet_count[src_ip] > self.DOS_THRESHOLD:
                self.logger.warning("ATTACCO DoS RILEVATO da %s! Blocco l'IP con regola a priorità 100.", src_ip)
                match = parser.OFPMatch(eth_type=ether_types.ETH_TYPE_IP, ipv4_src=src_ip)
                self.add_flow(datapath, 100, match, [])
                return

        dst = eth.dst
        src = eth.src
        dpid = datapath.id
        self.mac_to_port.setdefault(dpid, {})

        # Mostra sempre i log dei pacchetti in transito nel terminale
        self.logger.info("Packet-in [Switch DPID: %s] | Sorgente MAC: %s -> Destinazione MAC: %s | Porta In: %s", dpid, src, dst, in_port)

        # Apprendimento dell'indirizzo MAC
        self.mac_to_port[dpid][src] = in_port

        if dst in self.mac_to_port[dpid]:
            out_port = self.mac_to_port[dpid][dst]
        else:
            out_port = ofproto.OFPP_FLOOD

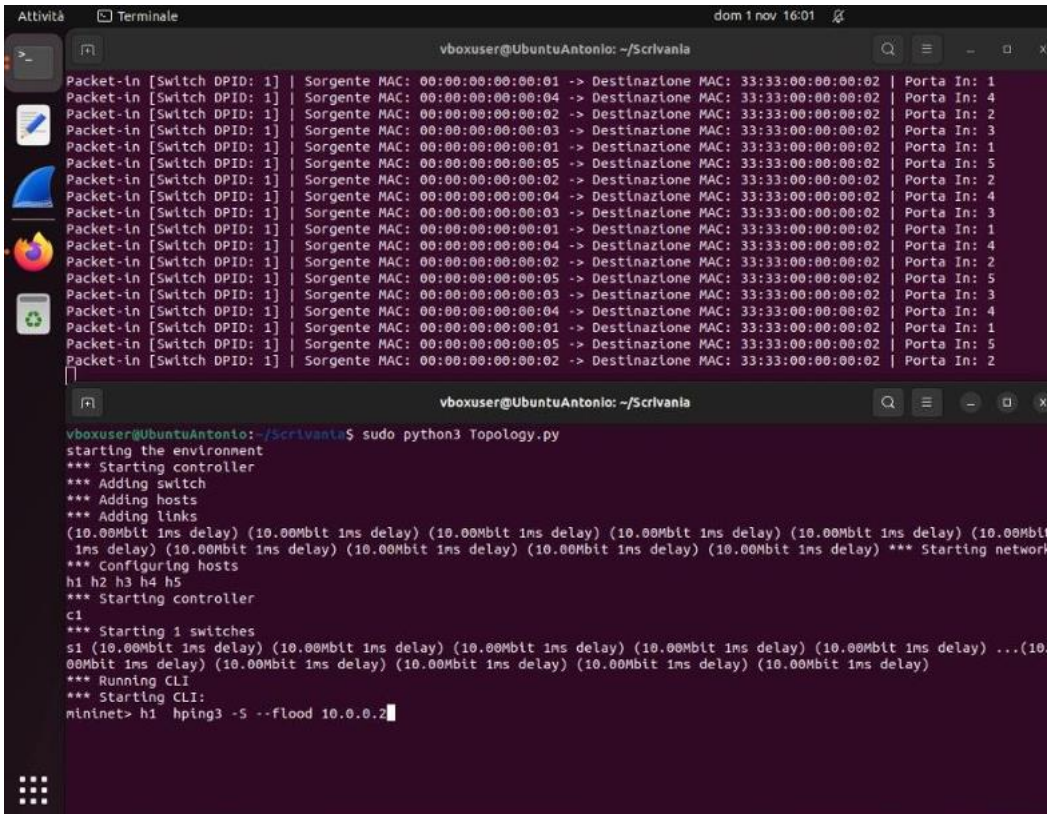
        actions = [parser.OFPActionOutput(out_port)]

        if out_port != ofproto.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
            if msg.buffer_id != ofproto.OFP_NO_BUFFER:
                self.add_flow(datapath, 1, match, actions, msg.buffer_id)
                return
            else:
                self.add_flow(datapath, 1, match, actions)

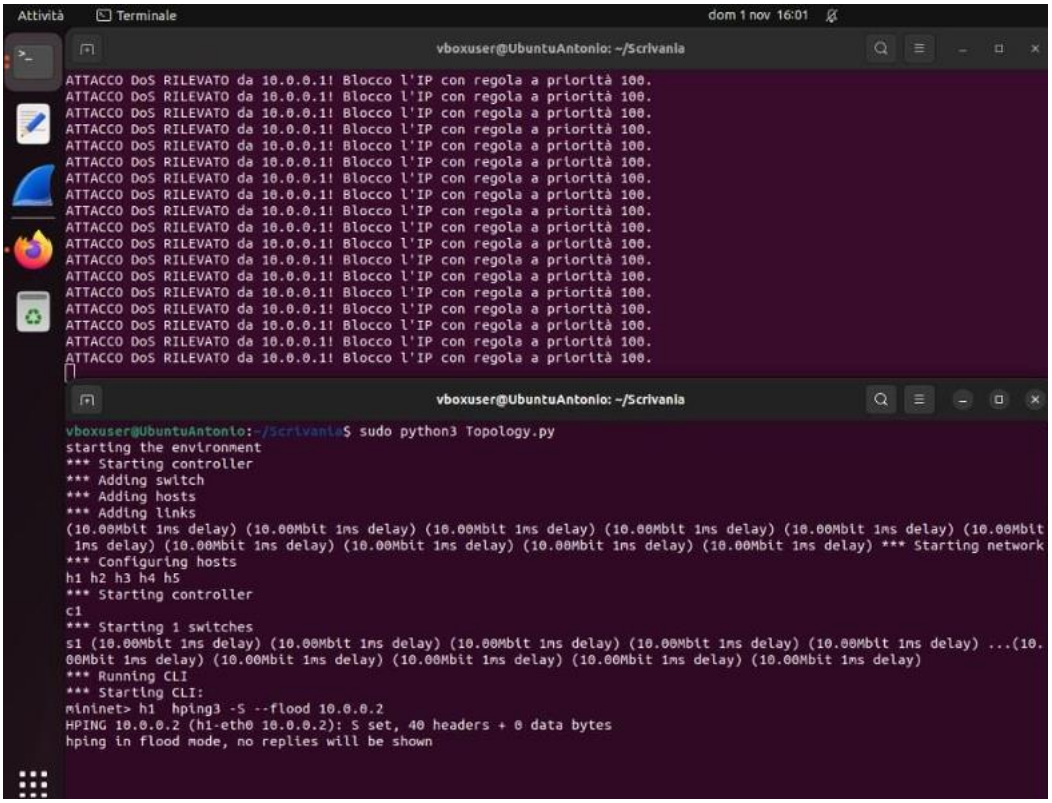
        data = None
        if msg.buffer_id == ofproto.OFP_NO_BUFFER:
            data = msg.data

        out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                                  in_port=in_port, actions=actions, data=data)
        datapath.send_msg(out)
```

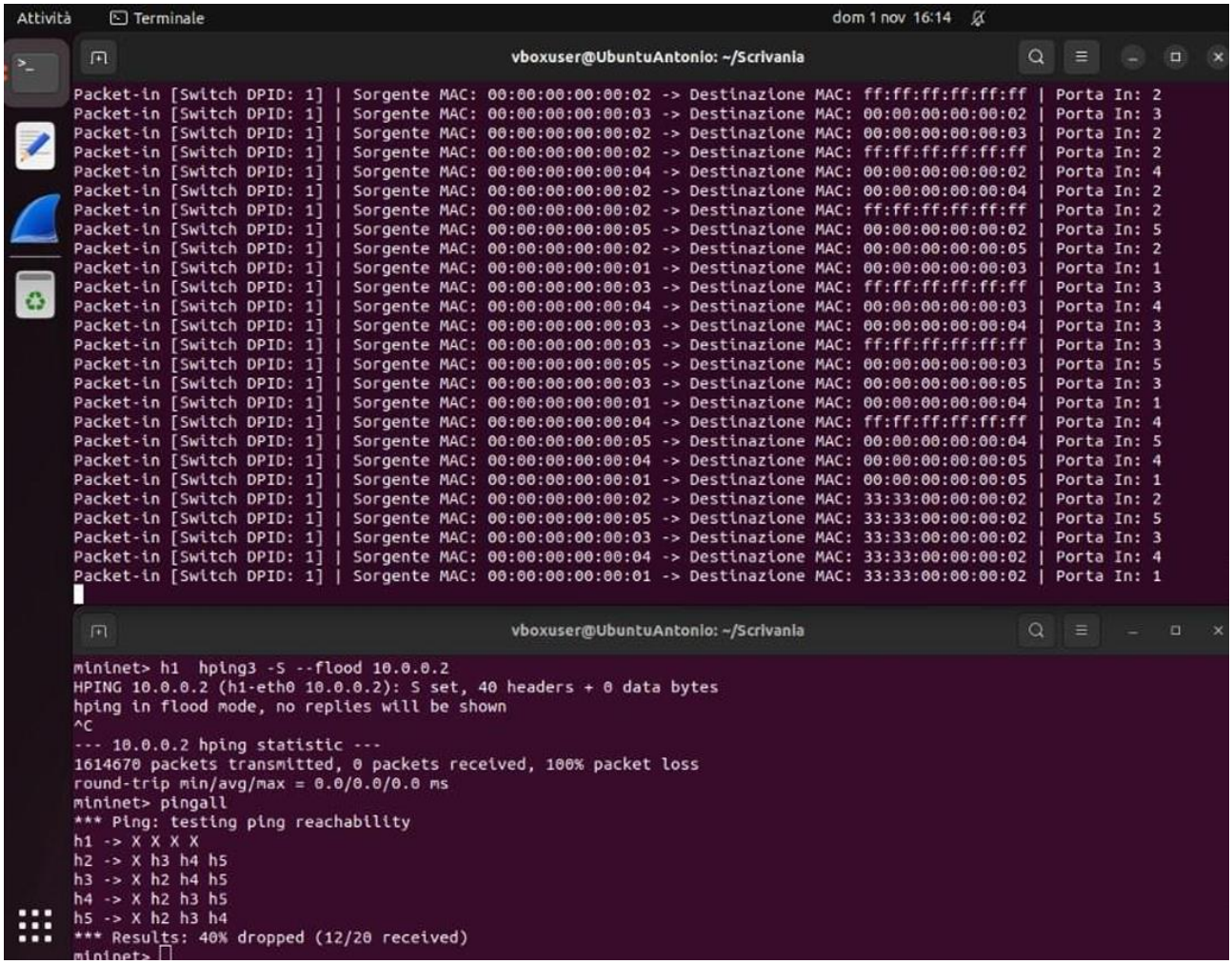
Esecuzione del test: Per prima cosa bisogna avviare lo script Topology.py e Controller_RYU.py, una volta sulla shell di mininet si digita il comando `h1 hping3 -S --flood 10.0.0.2`



Appena il nodo h1 supera la soglia di allarma fissata a 250 pacchetti l'attacco viene rilevato e segnalato. Subito dopo il nodo attaccante viene isolato dalla rete, l'attacco viene sventato e il traffico legittimo torna a funzionare correttamente come è possibile vedere dai log del controller



Nella prima shell si può vedere che il traffico legittimo è ripreso, in questo caso possiamo osservare i classici pacchetti inviati da un nodo all'altro così da costruire le tabelle di inoltro e aggiornare la topologia. Nella seconda shell possiamo vedere che il nodo è stato correttamente isolato. Il test pingall mostra che h1 non può raggiungere nessuno degli altri nodi e, viceversa, gli altri nodi non possono raggiungere h1. Al contempo il traffico tra i nodi h2 h3 h4 ed h5 procede regolarmente senza intoppi. Ciò dimostra che il meccanismo di mitigazione ha funzionato.



Per verificare la corretta installazione della regola di DROP basta digitare il comando `sh ovs-ofctl dump-flows s1 -O OpenFlow13`